

[À compléter : nom de l'institution]

[À compléter : département ou faculté]

Saveur : application de recettes de cuisine

Rapport académique de projet d'examen

Projet : Projet 5 – Application de Recettes de Cuisine

Cours : Développement JavaScript

Cours : Bases de données NoSQL

Étudiants : [À compléter : noms et prénoms des étudiants du groupe]

Groupe : [À compléter : numéro ou nom du groupe]

Enseignant : [À compléter : nom de l'enseignant]

Date de remise : [À compléter : date de remise officielle]

Année académique : 2025–2026

Rapport rédigé à partir de l'audit du dépôt Saveur

Résumé

Saveur est une application web full-stack de recettes de cuisine réalisée dans le cadre des cours de Développement JavaScript et de Bases de données NoSQL. Nous avons conçu une plateforme permettant de consulter un catalogue public de recettes, de filtrer les résultats par catégorie, temps et difficulté, de créer des recettes personnelles, de gérer des favoris, de planifier les repas de la semaine et de générer une liste de courses intelligente. Le frontend utilise React, TypeScript, Vite, Tailwind CSS et des composants dynamiques. Le backend repose sur Node.js, Express, MongoDB et Mongoose. L'authentification est assurée par un couple email/mot de passe, un hash bcrypt et un token JWT envoyé aux routes protégées. La base contient sept collections MongoDB liées par références et sous-documents. Les vérifications disponibles confirment que le typage TypeScript, les tests automatisés de la liste de courses et le build de production passent localement. Certaines validations restent manuelles, notamment les captures finales et les tests navigateur complets.

Mots-clés : JavaScript, React, Express, MongoDB, Mongoose, JWT, recettes, liste de courses.

Table des matières

Résumé	1
1 Introduction	6
2 Analyse des besoins	7
2.1 Présentation du problème	7
2.2 Objectifs fonctionnels	7
2.3 Objectifs non fonctionnels	9
2.4 Utilisateurs et rôles	10
2.5 Cas d'utilisation principaux	10
3 Conception technique	11
3.1 Architecture logicielle	11
3.2 Organisation générale du système	11
3.3 Modélisation NoSQL	12
3.4 Relations entre les collections	13
3.5 Justification des choix NoSQL	13
3.6 Authentification et autorisation	14
4 Implémentation	16
4.1 Stack technologique	16
4.2 Organisation du code	17
4.3 Fonctionnalités principales implémentées	17
4.3.1 Authentification	17
4.3.2 Recettes, ingrédients et étapes	18
4.3.3 Catalogue, favoris, profil et planning	18
4.3.4 Liste de courses intelligente	18
4.3.5 Images, langues et PWA	18
4.4 Extraits de code commentés	19
4.5 Interface utilisateur et expérience utilisateur	20

5	Tests et validation	21
5.1	Stratégie et environnement de test	21
5.2	Plan de tests	21
5.3	Résultats des tests	22
6	Difficultés rencontrées et solutions	24
6.1	Adaptation du projet au cahier des charges web	24
6.2	Modélisation des ingrédients	24
6.3	Catégorisation intelligente de la liste de courses	24
6.4	Protection des ressources personnelles	25
6.5	Cohérence de l'interface et navigation responsive	25
7	Améliorations possibles	26
8	Conclusion	27
	Références	28
A	Captures d'écran	29
B	Guide d'installation et d'exécution	34
C	Principales routes API	35
D	Répartition des tâches	36
E	Table de traçabilité interne	37

Table des figures

3.1	Architecture logicielle vérifiée de Saveur	11
3.2	Diagramme des collections MongoDB et de leurs relations principales . . .	13
A.1	Page d'accueil de Saveur avec recette mise en avant et catégories	30
A.2	Interface d'authentification utilisateur	30
A.3	Catalogue de recettes avec collections et filtres	31
A.4	Profil et tableau de bord de l'utilisateur authentifié	31
A.5	Planning hebdomadaire des repas	32
A.6	Liste de courses intelligente regroupée par rayons	32
A.7	Formulaire de création d'une recette avec image, ingrédients et étapes . . .	33
A.8	Détail d'une recette avec ingrédients et accès au mode cuisine	33

Liste des tableaux

2.1	Tableau des exigences fonctionnelles	8
3.1	Détail des collections MongoDB	12
3.2	Inventaire des routes API importantes	14
4.1	Technologies utilisées	16
5.1	Plan de tests	21
7.1	Améliorations futures	26
D.1	Répartition des tâches du groupe	36
E.1	Correspondance entre affirmations du rapport et preuves du dépôt	37

Chapitre 1

Introduction

Le présent projet s'inscrit dans le cadre des cours de Développement JavaScript et de Bases de données NoSQL. Ces deux enseignements visent à consolider les compétences nécessaires à la création d'une application web complète : interface dynamique côté client, serveur applicatif Node.js, API REST, persistance NoSQL et mécanismes de sécurité. Le document de projet exige explicitement une application frontend et backend, l'utilisation de JavaScript ES6+, Node.js/Express, MongoDB avec Mongoose, une authentification complète et une organisation claire selon les responsabilités du modèle, de la vue et du contrôleur.

Le sujet attribué à notre groupe est le Projet 5 : application de recettes de cuisine. Ce domaine implique une modélisation plus riche qu'un simple catalogue : une recette contient des informations descriptives, des ingrédients avec quantités et unités, des étapes de préparation, une catégorie, une cuisine, une image et un auteur. L'application doit aussi permettre à l'utilisateur de planifier ses repas et de transformer ce planning en liste de courses. Cette transformation est un élément central du sujet, car elle demande de parcourir plusieurs documents, de regrouper des ingrédients compatibles et de conserver l'état des articles déjà cochés.

Nous avons développé Saveur comme application web utilisable localement. L'utilisateur visite une page d'accueil orientée découverte, parcourt le catalogue, consulte les détails d'une recette, se connecte, crée ses propres recettes, ajoute des favoris, planifie ses repas et génère ses courses. Le tableau de bord personnel est placé dans la page de profil, où l'utilisateur retrouve ses indicateurs, son carnet de recettes, ses favoris, ses préférences de langue et les liens vers le planning.

Ce rapport suit la structure officielle demandée par le modèle de rapport. Il présente l'analyse des besoins, la conception technique et NoSQL, l'implémentation, la sécurité, les fonctionnalités, les tests, les difficultés rencontrées, les améliorations possibles et les annexes nécessaires pour la remise académique.

Chapitre 2

Analyse des besoins

2.1 Présentation du problème

La gestion de recettes devient rapidement complexe lorsque l'on ajoute les contraintes de la vie quotidienne. Un utilisateur peut avoir plusieurs recettes, différentes unités de mesure, des ingrédients répétés, des favoris, des repas à organiser dans la semaine et une liste de courses à préparer. Sans outil centralisé, les informations restent dispersées : les recettes sont consultées à un endroit, le planning est écrit ailleurs, et la liste de courses doit être reconstruite manuellement.

Saveur répond à ce problème en regroupant ces opérations dans une même application. Les recettes servent de source de données principale. Le planning hebdomadaire réutilise ces recettes. La liste de courses est générée à partir des ingrédients des repas planifiés. Les favoris et le carnet personnel permettent à l'utilisateur de retrouver rapidement les contenus importants. Le catalogue public facilite la découverte de recettes existantes et répond à l'exigence académique de filtrage.

2.2 Objectifs fonctionnels

Le tableau 2.1 présente les besoins fonctionnels, leur preuve dans le dépôt et leur état actuel. Les statuts distinguent les éléments implémentés, partiels, manquants ou non vérifiés.

TABLE 2.1: Tableau des exigences fonctionnelles

Exigence	Description	Utilisateur	Preuve dans le développement	Statut
Authentification	Inscription, connexion, session utilisateur	Visiteur, utilisateur	<code>server/controllers/authController.js</code> , <code>server/middleware/auth.js</code>	Implémenté
Mot de passe sécurisé	Hashage avant stockage	Utilisateur	<code>bcrypt.hash(password, 12)</code>	Implémenté
Recettes de cuisine	Création et lecture de recettes avec description, temps, catégorie et image	Utilisateur, visiteur	<code>server/models/Recipe.js</code> , <code>src/pages/CreateRecipe.tsx</code>	Implémenté
Ingrédients avec unités	Chaque ingrédient de recette possède quantité et unité	Utilisateur	<code>Recipe.ingredients[]</code>	Implémenté
Étapes de préparation	Étapes ordonnées avec titre et description	Utilisateur, visiteur	<code>Recipe.steps[]</code>	Implémenté
Catalogue public	Liste des recettes publiques	Visiteur, utilisateur	<code>GET /api/recipes</code> , <code>Catalog.tsx</code>	Implémenté
Filtres catalogue	Catégorie, temps maximal, difficulté et recherche	Visiteur, utilisateur	<code>listRecipes</code> , <code>getFilteredRecipes</code>	Implémenté
Favoris	Ajouter ou retirer une recette favorite	Utilisateur	<code>favoriteController.js</code>	Implémenté
Recettes personnelles	Lister les recettes créées par l'utilisateur	Utilisateur	<code>GET /api/recipes/mine</code>	Implémenté
Tableau de bord	Statistiques personnelles dans le profil	Utilisateur	<code>dashboardController.js</code> , <code>Profile.tsx</code>	Implémenté
Planning hebdomadaire	Planifier repas par date et type	Utilisateur	<code>MealPlan.js</code> , <code>Plan.tsx</code>	Implémenté
Liste de courses intelligente	Génération depuis le planning, agrégation et catégories de magasin	Utilisateur	<code>groceryListService.js</code> , <code>shoppingCategories.js</code>	Implémenté

Exigence	Description	Utilisateur	Preuve dans le dépôt	Statut
Téléversement d'image	Upload local, capture caméra, URL externe	Utilisateur	<code>uploadRoutes.js</code> , <code>CreateRecipe.tsx</code>	Implémenté localement
Internationalisation	Interface en anglais, français, espagnol, italien	Tous	<code>src/services/i18n.ts</code>	Partiel à vérifier
Mode PWA	Manifest, service worker, page hors ligne	Tous	<code>public/manifest.webmanifest</code> , <code>public/sw.js</code>	Manifeste implémentée
Édition de recette	Modifier une recette après création	Utilisateur	Aucun point PUT/PATCH <code>/api/recipes/:id</code> trouvé	Manquant
Administration visible	Interface administrateur	Admin	Rôle <code>admin</code> dans <code>User.js</code> , pas d'écran admin vérifié	Partiel

2.3 Objectifs non fonctionnels

Sur le plan de la sécurité, Saveur utilise `bcrypt` pour éviter le stockage des mots de passe en clair et `JWT` pour protéger les routes sensibles. Le middleware `requireAuth` vérifie le jeton et charge l'utilisateur avant d'autoriser l'accès. Certaines protections d'appartenance existent : la suppression d'une recette vérifie que l'utilisateur est propriétaire ou administrateur, et les ressources de planning, favoris, dashboard et courses sont filtrées par `req.user._id`.

La maintenabilité est favorisée par une organisation claire du code. Le backend sépare les modèles `Mongoose`, routes `Express`, contrôleurs, middleware, services et utilitaires. Le frontend sépare les pages `React`, services `API`, types `TypeScript`, composants partagés et utilitaires. Cette structure facilite l'audit et répond à la consigne de séparation des responsabilités.

La performance est traitée de manière simple mais suffisante pour un MVP académique. Les listes de recettes sont limitées à 100 éléments côté backend. Les filtres sont appliqués en base pour la catégorie, la difficulté, la recherche textuelle et le temps maximal. Le catalogue applique ensuite une pagination côté interface. Un index texte existe sur le titre et la description des recettes.

L'ergonomie a fait l'objet de corrections successives : navigation desktop sans débordement

dement, page d'accueil séparée du dashboard, profil comme espace personnel, catalogue paginé, liste de courses regroupée par sections de magasin et formulaire de recette avec options d'image. L'accessibilité n'a pas été audité par un outil spécialisé ; nous la classons donc comme partiellement vérifiée. La fiabilité repose sur la gestion des erreurs Express et sur des messages JSON lorsque les requêtes échouent. Le service worker ajoute une base PWA, mais l'édition hors ligne et la synchronisation différée ne sont pas implémentées.

2.4 Utilisateurs et rôles

Le visiteur non connecté peut consulter les pages publiques, notamment l'accueil, le catalogue et les détails des recettes publiques. Il ne peut pas créer de recette, ajouter de favori, planifier de repas ou générer une liste de courses. Les actions protégées redirigent vers la page de profil/connexion ou sont bloquées par l'API.

L'utilisateur authentifié peut créer des recettes, consulter son carnet, gérer ses favoris, remplir le planning, générer la liste de courses, cocher les articles et accéder au dashboard personnel. Les données personnelles sont liées à son identifiant MongoDB.

Le rôle administrateur existe dans le modèle `User` et dans le token JWT. Le backend autorise déjà la suppression d'une recette par un administrateur. Toutefois, aucune interface d'administration complète n'a été vérifiée dans le dépôt. Nous considérons donc l'administration comme une possibilité technique partielle, non comme une fonctionnalité complète.

2.5 Cas d'utilisation principaux

Les principaux parcours sont l'inscription et la connexion, la consultation du catalogue, le filtrage des recettes, la création d'une recette personnelle, l'ajout aux favoris, la planification d'un repas, la génération de la liste de courses et la gestion du profil. La création de recette enregistre les ingrédients avec quantité et unité. La génération des courses lit les recettes planifiées sur la semaine, agrège les ingrédients compatibles, conserve les articles personnalisés et présente les résultats par catégories de magasin.

Chapitre 3

Conception technique

3.1 Architecture logicielle

L'application suit une architecture trois tiers. La couche cliente est une application React/Vite. Elle consomme une API REST exposée par Express. La couche de données est MongoDB, manipulée via Mongoose. En production locale, Express peut aussi servir le dossier `dist` généré par Vite.

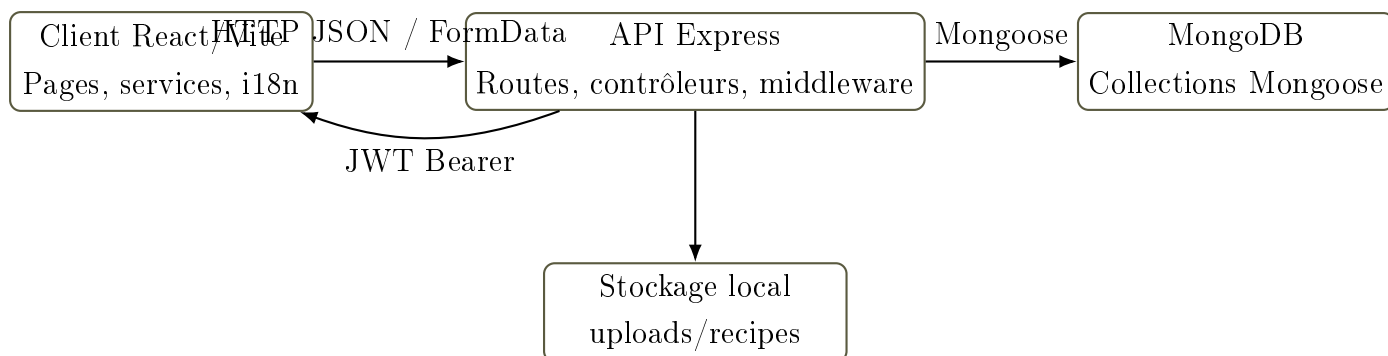


FIGURE 3.1 – Architecture logicielle vérifiée de Saveur

3.2 Organisation générale du système

Le frontend porte la responsabilité de l’affichage, de la navigation, de la gestion locale de la session, de la traduction des libellés et des interactions utilisateur. Les appels API sont regroupés dans `src/services`. Les pages principales sont `Home.tsx`, `Catalog.tsx`, `RecipeDetail.tsx`, `CreateRecipe.tsx`, `Plan.tsx`, `Groceries.tsx` et `Profile.tsx`.

Le backend centralise les règles métier. Les routes sont montées dans `server/index.js` sous les préfixes `/api/auth`, `/api/categories`, `/api/ingredients`, `/api/recipes`, `/api/favorites`, `/api/meal-plans`, `/api/groceries`, `/api/dashboard` et `/api/uploads`. Les contrôleurs manipulent les modèles Mongoose et renvoient des réponses JSON. La

base conserve les comptes, catégories, ingrédients canoniques, recettes, favoris, entrées de planning et listes de courses. Les images téléversées sont stockées localement dans `uploads/recipes`.

3.3 Modélisation NoSQL

Le modèle NoSQL combine références et sous-documents. Les références permettent de relier les utilisateurs, catégories, recettes, favoris et plans. Les sous-documents conviennent aux ingrédients d'une recette, aux étapes et aux articles de courses, car ces données sont généralement chargées avec leur document parent.

TABLE 3.1: Détail des collections MongoDB

Collection	Rôle	Champs principaux	Contraintes et relations
<code>users</code>	Comptes utilisateur	<code>name</code> , <code>email</code> , <code>passwordHash</code> , <code>role</code> , <code>photoURL</code> , <code>preferredLanguage</code>	Email unique, rôle <code>user/admin</code> , langue <code>en/fr/es/it</code> . Référencé par recettes, favoris, plans et listes.
<code>categories</code>	Collections de recettes	<code>name</code> , <code>slug</code> , <code>image</code>	Nom unique, <code>slug</code> obligatoire. Référencé par <code>recipes</code> .
<code>ingredients</code>	Catalogue canonique d'ingrédients	<code>name</code> , <code>defaultUnit</code> , <code>category</code> , <code>caloriesPerUnit</code>	Nom unique, catégorie de courses stable, unité par défaut.
<code>recipes</code>	Recettes et contenu culinaire	<code>title</code> , <code>description</code> , <code>temps</code> , <code>difficulté</code> , <code>portions</code> , <code>cuisine</code> , <code>image</code> , <code>visibilité</code> , <code>ingredients []</code> , <code>steps []</code>	Référence <code>user</code> , référence <code>category</code> , ingrédients et étapes embarqués, index texte titre/description.
<code>favorites</code>	Favoris utilisateur	<code>user</code> , <code>recipe</code>	Index unique <code>user + recipe</code> . Relation de jonction.
<code>mealplans</code>	Planning hebdomadaire	<code>user</code> , <code>date</code> , <code>mealType</code> , <code>recipe</code>	Date au format ISO, type de repas <code>enum</code> , index unique <code>user + date + mealType</code> .

Collection	Rôle	Champs principaux	Contraintes et relations
shoppinglists	Courses par semaine	user, weekStart, items []	Index unique user + weekStart. Articles embarqués : nom, quantité, unité, catégorie, source, checked.

3.4 Relations entre les collections

Un utilisateur peut créer plusieurs recettes. Une catégorie peut classer plusieurs recettes. Les favoris constituent une relation plusieurs-à-plusieurs entre utilisateurs et recettes. La collection **favorites** joue le rôle de document de jonction, avec un index unique pour éviter qu'un utilisateur ajoute deux fois la même recette. Le planning relie un utilisateur à une recette pour une date et un type de repas. La liste de courses est associée à l'utilisateur et à un début de semaine. Ses articles sont embarqués, car ils représentent le résultat calculé et modifiable d'une génération.

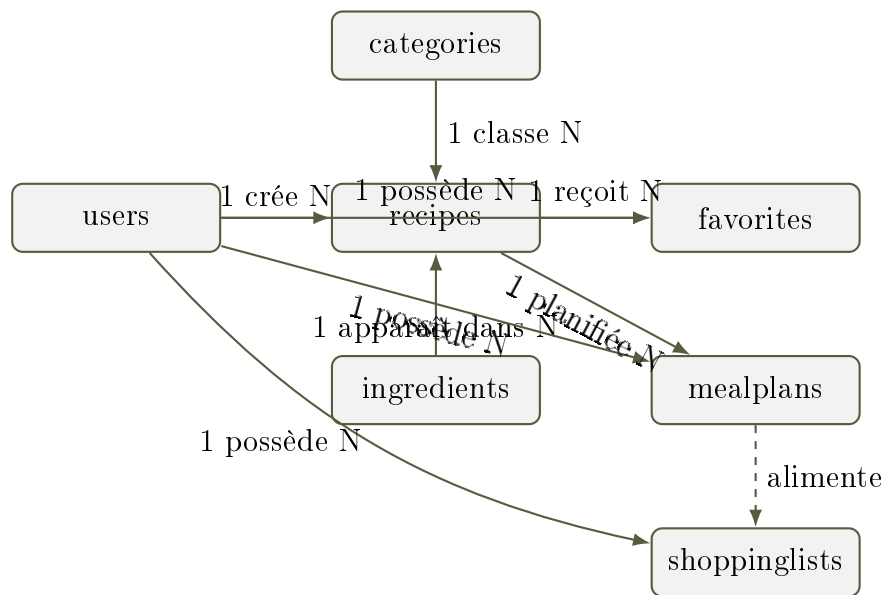


FIGURE 3.2 – Diagramme des collections MongoDB et de leurs relations principales

3.5 Justification des choix NoSQL

MongoDB convient à ce projet parce qu'une recette est naturellement un document riche. Elle contient des métadonnées, une catégorie, une visibilité, des ingrédients et des étapes. Les ingrédients et les étapes n'ont pas besoin d'être interrogés indépendamment dans la majorité des cas ; leur embarquement réduit donc le nombre de requêtes lors de

l’affichage d’une fiche recette. Les références restent nécessaires pour les entités réutilisables. Les utilisateurs, catégories, recettes planifiées et favoris sont reliés par identifiants. La collection `ingredients` apporte une couche de normalisation utile pour la liste de courses, car chaque ingrédient canonique possède une unité par défaut et une catégorie de magasin stable.

3.6 Authentification et autorisation

Lors de l’inscription, le client envoie nom, email et mot de passe à `POST /api/auth/register`. Le contrôleur vérifie la présence des champs et impose un mot de passe d’au moins six caractères. Il recherche ensuite un éventuel compte existant par email. Si l’email est libre, il appelle `bcrypt.hash(password, 12)` et crée l’utilisateur avec le hash. Le serveur retourne un JWT et un objet utilisateur sérialisé.

Lors de la connexion, `POST /api/auth/login` vérifie l’email et compare le mot de passe saisi avec `bcrypt.compare`. Si la comparaison réussit, `createToken` crée un JWT contenant `sub` et `role`, avec une expiration de sept jours. Côté frontend, `AuthContext.tsx` stocke la session dans `localStorage` sous la clé `saveur_session`. Le service `apiFetch` lit ensuite cette session et ajoute l’en-tête `Authorization: Bearer <token>`.

Le middleware `requireAuth` extrait le token, vérifie sa signature avec `JWT_SECRET`, charge l’utilisateur sans `passwordHash` et place l’objet dans `req.user`. Les routes protégées peuvent alors utiliser `req.user._id` pour limiter les données à l’utilisateur connecté.

TABLE 3.2: Inventaire des routes API importantes

Méthode	Route	Rôle	Auth	Fichier
POST	<code>/api/auth/register</code>	Créer un compte	Non	<code>authRoutes.js</code>
POST	<code>/api/auth/login</code>	Connecter un utilisateur	Non	<code>authRoutes.js</code>
GET	<code>/api/auth/me</code>	Récupérer la session	Oui	<code>authRoutes.js</code>
GET	<code>/api/categories</code>	Lister les catégories	Non	<code>categoryRoutes.js</code>
GET	<code>/api/ingredients</code>	Chercher les ingrédients	Non	<code>ingredientRoutes.js</code>
GET	<code>/api/recipes</code>	Catalogue public filtré	Optionnelle	<code>recipeRoutes.js</code>
GET	<code>/api/recipes/:id</code>	Détail recette	Optionnelle	<code>recipeRoutes.js</code>
POST	<code>/api/recipes</code>	Créer une recette	Oui	<code>recipeRoutes.js</code>
GET	<code>/api/recipes/mine</code>	Recettes personnelles	Oui	<code>recipeRoutes.js</code>
DELETE	<code>/api/recipes/:id</code>	Supprimer une recette	Oui	<code>recipeRoutes.js</code>
GET	<code>/api/favorites</code>	Lister les favoris	Oui	<code>favoriteRoutes.js</code>
POST	<code>/api/favorites/:recipeId/toggle</code>	Ajouter/supprimer favori	Oui	<code>favoriteRoutes.js</code>
GET	<code>/api/meal-plans</code>	Lire le planning	Oui	<code>mealPlanRoutes.js</code>
POST	<code>/api/meal-plans</code>	Ajouter/remplacer repas	Oui	<code>mealPlanRoutes.js</code>

Méthode	Route	Rôle	Auth	Fichier
GET	/api/groceries	Générer/lire les courses	Oui	groceryRoutes.js
POST	/api/groceries/custom	Ajouter article personnalisé	Oui	groceryRoutes.js
PATCH	/api/groceries/:id	Modifier/décocher article	Oui	groceryRoutes.js
GET	/api/dashboard	Statistiques profil	Oui	dashboardRoutes.js
POST	/api/uploads/recipeImage	Envoyer image recette	Oui	uploadRoutes.js

Chapitre 4

Implémentation

4.1 Stack technologique

TABLE 4.1: Technologies utilisées

Technologie	Version	Rôle	Justification
React	^19.0.1	Interface utilisateur	Composants dynamiques et SPA.
TypeScript	~5.8.2	Typage frontend	Limiter les erreurs de structure de données.
Vite	^6.2.3	Build et serveur dev	Démarrage rapide et build optimisé.
Tailwind CSS	^4.1.14	Styles	Interface responsive et cohérente.
Node.js	24.14.0 ob- servé	Runtime backend	Exécution JavaScript serveur.
Express	^4.21.2	API REST	Routage HTTP et middleware.
MongoDB	Local	Base NoSQL	Documents flexibles et relations par références.
Mongoose	^8.9.5	ODM	Schémas, validations et requêtes MongoDB.
bcryptjs	^2.4.3	Hashage	Sécurisation des mots de passe.
jsonwebtoken	^9.0.2	JWT	Sessions stateless côté API.
multer	^2.2.0	Upload	Réception des images de recette.

Technologie	Version	Rôle	Justification
Vitest	^4.1.5	Tests	Tests unitaires du service de courses.

4.2 Organisation du code

Listing 4.1 – Arborescence commentée du projet

```
server/
  config/db.js # connexion Mongoose
  controllers/ # logique des endpoints REST
  data/seedData.js # categories, ingredients, recettes, comptes demo
  middleware/auth.js # JWT, optionalAuth, requireRole
  models/ # schemas User, Recipe, Category...
  routes/ # definition des routes Express
  services/groceryListService.js # generation et aggregation des courses
  utils/shoppingCategories.js # categories et alias d'ingredients
src/
  components/ # layout, selecteur de langue
  pages/ # Home, Catalog, Plan, Groceries, Profile...
  services/ # appels API, auth, i18n
  utils/ # icones de categories, categories de courses
  types.ts # types partages frontend
public/
  manifest.webmanifest # PWA manifest
  sw.js # service worker
  offline.html # fallback hors ligne
```

4.3 Fonctionnalités principales implémentées

4.3.1 Authentification

L'authentification est implémentée côté backend avec `authController.js`. L'inscription crée un utilisateur après vérification de l'email et hashage du mot de passe. La connexion compare le mot de passe avec le hash existant. Le frontend expose le formulaire de connexion et d'inscription dans `Profile.tsx`. Les champs de démonstration sont préremplis avec le compte `chef@savoir.local`, ce qui facilite les tests locaux. Cette donnée est explicitement présente dans le seed et le README ; elle doit être changée ou retirée dans un environnement de production.

4.3.2 Recettes, ingrédients et étapes

La création de recette est disponible pour les utilisateurs connectés. Le formulaire contient les informations générales, la catégorie, la cuisine, l'image, les temps, le niveau de difficulté, les portions, les ingrédients et les étapes. Côté serveur, `createRecipe` vérifie que la catégorie existe, filtre les étapes vides et normalise les ingrédients. Une recette privée ne peut être consultée que par son propriétaire ou par un administrateur. La suppression exige que l'utilisateur soit propriétaire de la recette ou administrateur. En revanche, l'édition complète d'une recette existante n'a pas été vérifiée dans le dépôt.

4.3.3 Catalogue, favoris, profil et planning

Le catalogue utilise `GET /api/recipes`. Les filtres vérifiés sont la catégorie, la difficulté, le temps maximal et la recherche texte. Les favoris sont stockés dans une collection dédiée, ce qui évite d'embarquer une longue liste dans le document utilisateur. Le profil charge les recettes personnelles, les favoris et les statistiques du dashboard. Le planning est stocké dans `mealplans`; chaque document contient l'utilisateur, la date, le type de repas et la recette. Un index unique empêche deux recettes d'occuper le même créneau pour le même utilisateur.

4.3.4 Liste de courses intelligente

Le contrôleur `generateGroceryList` récupère les repas planifiés entre `weekStart` et `weekStart + 6 jours`. Il collecte les identifiants des ingrédients canoniques, charge leurs catégories, récupère l'ancienne liste pour conserver les éléments cochés et appelle `buildGeneratedGroceryItems`. Les articles personnalisés existants sont ensuite réintégrés.

Les catégories de magasin sont stables : `DAIRY`, `PRODUCE`, `PASTA_GRAINS`, `PANTRY`, `MEAT_PROTEIN`, `BAKERY`, `FROZEN`, `BEVERAGES` et `OTHER`. Les libellés visibles sont traduits côté frontend. Des alias permettent de reconnaître des variantes comme `Greek yogurt`, `yaourt grec`, `tomatoes`, `tagliatelle` ou `walnuts`. Lorsqu'aucune correspondance fiable n'est trouvée, l'article tombe dans `OTHER`.

4.3.5 Images, langues et PWA

Le formulaire conserve trois options d'image : URL externe, upload local et capture caméra. L'endpoint `POST /api/uploads/recipe-image` accepte un champ multipart `photo`, limite la taille à 5 Mo et autorise JPG, PNG et WebP. L'interface contient des dictionnaires pour l'anglais, le français, l'espagnol et l'italien. Le projet contient aussi un manifest, une icône, un service worker et une page `offline.html`. Nous considérons donc la PWA comme une base installable, non comme une application offline-first complète.

4.4 Extraits de code commentés

Listing 4.2 – Middleware d’authentification – server/middleware/auth.js

```
const header = req.headers.authorization || '';
const token = header.startsWith('Bearer ') ? header.slice(7) : null;
if (!token) {
  return res.status(401).json({ message: 'Authentication required.' });
}
const payload = jwt.verify(token, process.env.JWT_SECRET);
const user = await User.findById(payload.sub).select('-passwordHash');
```

Cet extrait montre comment les routes privées refusent les requêtes non authentifiées, vérifient le token et chargent l'utilisateur sans exposer le hash du mot de passe.

Listing 4.3 – Normalisation d’ingrédients – server/controllers/recipeController.js

```
const ingredient = await Ingredient.findOneAndUpdate(
  { name: new RegExp(`^${name.replace(/[\.*?~${}()|[\]\\"/g, '\\$&')}$`, 'i') },
  {
    $setOnInsert: { name, defaultUnit: unit },
    $set: { category: suggestShoppingCategory(name, item.category) },
  },
  { new: true, upsert: true }
);
```

Cet extrait prouve que les ingrédients ne sont pas seulement manipulés dans l’interface. Le backend crée ou met à jour un ingrédient canonique, en tenant compte d’une suggestion de catégorie de courses.

Listing 4.4 – Agrégation des courses – server/services/groceryListService.js

```
const key = groceryItemKey({ name: recipeIngredient.name, unit });
const previous = previousByKey.get(key);
const current = itemMap.get(key) || {
  name: recipeIngredient.name,
  quantity: 0,
  unit,
  category: suggestShoppingCategory(recipeIngredient.name, canonicalIngredient?.category),
  checked: Boolean(previous?.checked),
  source: 'generated',
};
current.quantity += Number(recipeIngredient.quantity) || 0;
itemMap.set(key, current);
```

Cet extrait montre la logique essentielle : normaliser la clé d'article, conserver l'état coché précédent, résoudre la catégorie et additionner les quantités compatibles.

4.5 Interface utilisateur et expérience utilisateur

La direction visuelle utilise des tons olive et neutres, des titres serif, des cartes arrondies et des ombres discrètes. La navigation principale contient Accueil, Catalogue, Planning et Courses. Le profil est accessible par l'avatar, car il contient le dashboard personnel. La page d'accueil sert à la découverte : recette du jour, catégories et recommandations. Le catalogue propose les collections, les filtres et la pagination. Le profil regroupe l'identité utilisateur, les statistiques, le carnet, les favoris et les préférences. Le planning se concentre sur la semaine et les types de repas. La liste de courses suit une organisation pratique par section de magasin. Le détail recette sépare les ingrédients et les étapes, avec un mode cuisine focalisé.

Chapitre 5

Tests et validation

5.1 Stratégie et environnement de test

La stratégie de test combine contrôles automatisés et validation manuelle. Les contrôles automatisés présents couvrent le service de liste de courses : reconnaissance d’alias, agrégation de quantités, conservation de l’état coché et ajout personnalisé. Le typage TypeScript sert de contrôle statique via `npm run lint`. Le build de production vérifie que l’application peut être compilée avec Vite. Aucun test end-to-end navigateur complet n’a été trouvé dans le dépôt.

Les commandes ont été exécutées dans le dossier C:

Users

migue

JS_NoSQL

Saveur--Recipe-Meal-Planner. L’environnement observé utilise Windows, Node.js 24.14.0 dans le terminal utilisateur, MongoDB local sur 127.0.0.1:27017, Vite pour le frontend et Express pour le backend.

5.2 Plan de tests

TABLE 5.1: Plan de tests

ID	Fonction	Préconditions	Étapes	Résultat attendu	Statut
T01	Inscription	API active	Créer un compte valide	Token et utilisateur retournés	À valider
T02	Connexion demo	Seed actif	Utiliser <code>chef@saveur.local</code>	Session créée	À valider

ID	Fonction	Préconditions	Étapes	Résultat attendu	Statut
T03	Mauvais mot de passe	Compte existant	Mot de passe incorrect	Erreur 401	À valider
T04	Route protégée	Pas de token	Appeler <code>/api/auth/me</code>	Erreur 401	À valider
T05	Catalogue	Seed actif	Ouvrir <code>/catalog</code>	Recettes affichées	À valider
T06	Filtre catégorie	Catalogue chargé	Choisir une collection	Résultats filtrés	À valider
T07	Filtre temps	Catalogue chargé	Choisir un temps maximal	Recettes sous limite	À valider
T08	Filtre difficulté	Catalogue chargé	Choisir une difficulté	Recettes filtrées	À valider
T09	Création recette	Connecté	Remplir formulaire	Recette créée	À valider
T10	Favori	Connecté	Cliquer sur coeur	Favori modifié	À valider
T11	Planning	Connecté	Choisir repas et recette	Entrée créée	À valider
T12	Courses agrégation	Tests unitaires	Exécuter <code>npm run test</code>	Quantités agrégées	OK
T13	Catégorie courses	Tests unitaires	Exécuter <code>npm run test</code>	Alias reconnus	OK
T14	État coché	Tests unitaires	Régénérer une liste simulée	<code>checked</code> conservé	OK
T15	Build production	Dépendances installées	Exécuter <code>npm run build</code>	<code>dist</code> généré	OK
T16	Responsive	Navigateur	Tester mobile/tablette/desktop	Pas de débordement	À valider

5.3 Résultats des tests

Les commandes suivantes ont été exécutées avec succès le 12 juillet 2026 : `npm.cmd run lint` a terminé `tsc -noEmit` sans erreur ; `npm.cmd run test` a exécuté 1 fichier de test avec 4 tests réussis ; `npm.cmd run build` a transformé 2102 modules et généré le dossier `dist`. Ces résultats confirment que le typage, le service de courses et la compilation de

production sont opérationnels.

Chapitre 6

Difficultés rencontrées et solutions

6.1 Adaptation du projet au cahier des charges web

Le projet avait initialement une orientation mobile. Le sujet de l'enseignant impose une application web avec frontend, backend Node.js/Express et MongoDB. Nous avons donc recentré l'application sur React/Vite côté client, Express côté serveur et MongoDB/Mongoose côté données. Le résultat est une structure full-stack cohérente avec les exigences, mais les captures finales doivent encore remplacer les anciennes captures annotées.

6.2 Modélisation des ingrédients

La génération d'une liste de courses exige de connaître les quantités et les unités. Une simple liste de noms d'ingrédients aurait été insuffisante. La solution retenue consiste à embarquer dans chaque recette les ingrédients avec `name`, `quantity`, `unit` et `optional`, tout en gardant une collection `ingredients` réutilisable. La limite restante est l'absence de conversion entre unités différentes.

6.3 Catégorisation intelligente de la liste de courses

Une liste de courses utile doit être organisée selon les sections du magasin. La catégorie ne devait pas être décidée uniquement dans le composant React. Nous avons donc ajouté des catégories stables côté domaine, une table d'alias et une fonction `suggestShoppingCategory`. Les ingrédients et articles de courses conservent une catégorie interne. Les inconnus restent classés dans `OTHER` jusqu'à correction manuelle ou enrichissement du catalogue.

6.4 Protection des ressources personnelles

Les recettes personnelles, favoris, plans et listes de courses ne doivent pas être mélangés entre utilisateurs. Les routes protégées utilisent `requireAuth`. Les requêtes MongoDB filtrent généralement par `req.user._id`. La suppression de recette vérifie la propriété ou le rôle administrateur. Un audit de sécurité complet reste nécessaire avant production ; l'utilisation de `localStorage` pour le JWT est acceptable pour le projet, mais des cookies HTTP-only seraient préférables en production.

6.5 Cohérence de l'interface et navigation responsive

Les corrections récentes ont porté sur le navbar desktop, la page d'accueil, le profil et la liste de courses. L'application avait évolué rapidement depuis un prototype ; certaines responsabilités d'écran étaient mélangées. Le dashboard a été déplacé dans le profil, l'accueil a été orienté découverte, la navigation a été simplifiée et la liste de courses a été restructurée en sections. Les captures finales doivent encore confirmer visuellement chaque écran dans l'état de soumission.

Chapitre 7

Améliorations possibles

TABLE 7.1: Améliorations futures

Priorité	Limitation actuelle	ac-	Solution proposée	Bénéfice attendu
Court terme	Peu de tests automatisés		Ajouter tests API et composants React	Réduire les régressions
Court terme	Édition de recette non vérifiée		Ajouter endpoint <code>PATCH /api/recipes/:id</code> et écran d'édition	Gestion complète des recettes
Moyen terme	Stockage image local		Migrer vers Cloudinary, S3 ou équivalent	Persistance après déploiement
Moyen terme	JWT stocké dans <code>localStorage</code>		Utiliser cookies HTTP-only et durcissement CSRF	Sécurité renforcée
Moyen terme	Internationalisation à valider		Relire tous les libellés et caractères	Interface plus professionnelle
Long terme	Pas de conversion d'unités		Ajouter normalisation g/kg/ml/l	Courses plus exactes
Long terme	Pas d'avis utilisateurs		Ajouter notation, commentaires et modération	Découverte enrichie
Long terme	PWA limitée		Ajouter cache de données et synchronisation différée	Hors ligne plus complet

Chapitre 8

Conclusion

Nous avons atteint l'objectif principal : produire une application web full-stack de recettes conforme à l'esprit du sujet. Saveur contient un frontend dynamique, un backend Express, une base MongoDB avec sept collections, une authentification par JWT et bcrypt, des recettes avec ingrédients et étapes, un catalogue filtrable, des favoris, un profil/dashboard, un planning hebdomadaire et une liste de courses intelligente.

Le projet nous a permis de renforcer nos compétences en JavaScript moderne, en architecture REST, en modélisation NoSQL et en séparation des responsabilités. La mise en place de Mongoose nous a aidés à structurer les documents tout en profitant de la flexibilité MongoDB. L'authentification nous a permis de manipuler les notions de hashage, token, middleware et routes protégées. Côté frontend, React et TypeScript ont facilité l'organisation en pages, services et composants.

L'application n'est pas encore un produit final complet. Les tests navigateur doivent être documentés, les captures finales doivent être produites, l'édition des recettes reste à ajouter, et les améliorations de sécurité seront nécessaires avant une mise en production réelle. Malgré ces limites, Saveur constitue une base fonctionnelle et défendable pour la remise académique et la soutenance.

Bibliographie

- [1] Document d'examen : *PROJETS D'EXAMEN DE DEVELOPPEMENT JAVASCRIPT & BASE DE DONNEES NOSQL*, fourni par l'enseignant.
- [2] Modèle de rapport : *MODEL DU RAPPORT PROJETS D'EXAMEN DE DEVELOPPEMENT JAVASCRIPT & BASE DE DONNEES NOSQL*, fourni par l'enseignant.
- [3] Node.js Documentation, <https://nodejs.org/docs/latest/api/>.
- [4] Express Documentation, <https://expressjs.com/>.
- [5] MongoDB Manual, <https://www.mongodb.com/docs/>.
- [6] Mongoose Documentation, <https://mongoosejs.com/docs/>.
- [7] React Documentation, <https://react.dev/>.
- [8] Vite Documentation, <https://vite.dev/>.
- [9] JSON Web Tokens, <https://jwt.io/introduction>.

Annexe A

Captures d'écran

Le modèle de rapport exige au moins six captures. Les figures A.1 à A.8 présentent les principaux écrans vérifiés de l'application dans leur état final. Elles montrent respectivement l'accueil, l'authentification, le catalogue, le profil, le planning, la liste de courses, le formulaire de création et la fiche détaillée d'une recette.

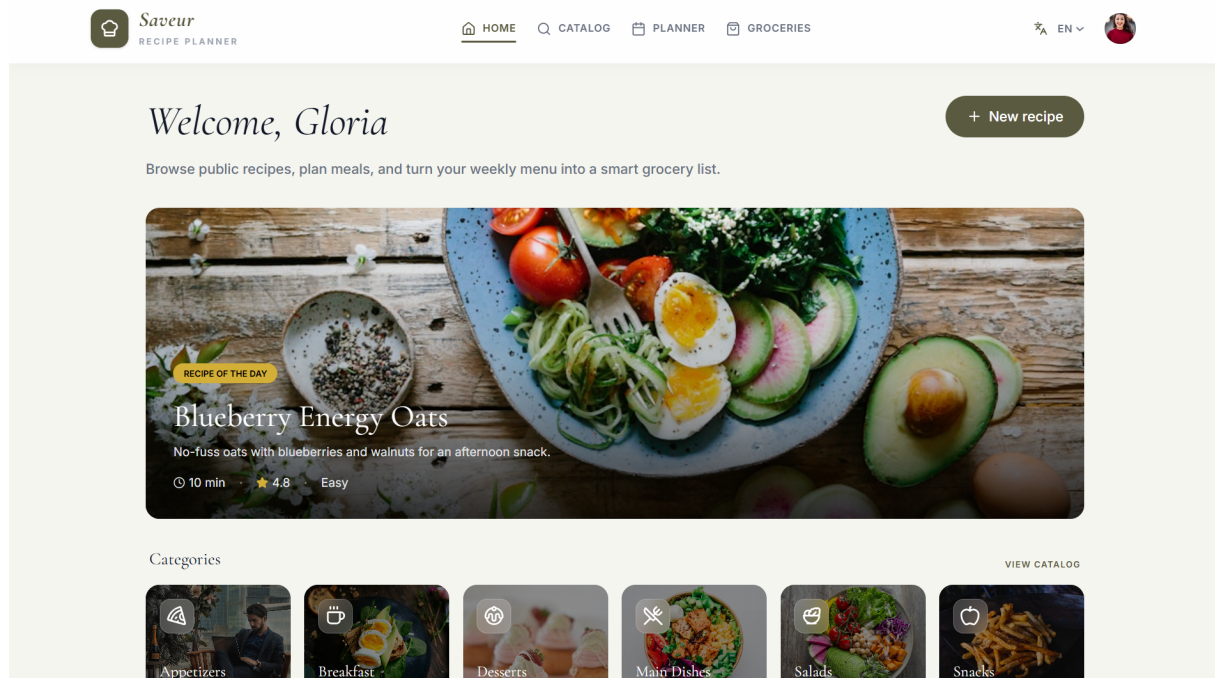


FIGURE A.1 – Page d'accueil de Saver avec recette mise en avant et catégories

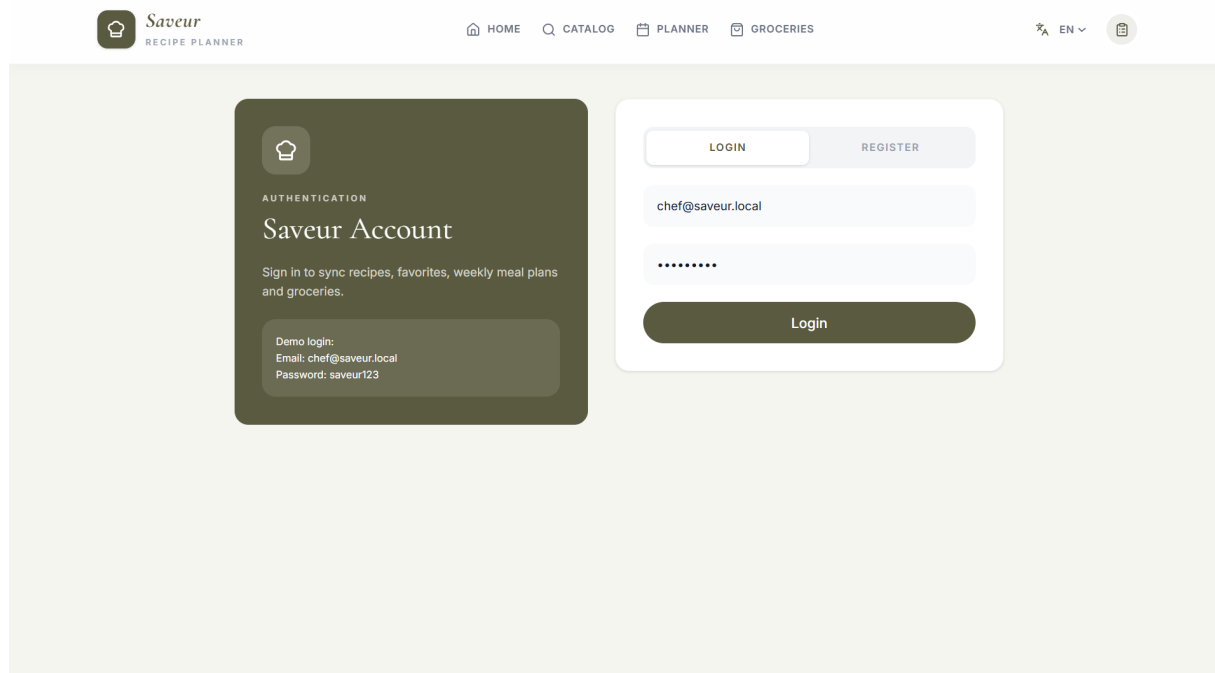


FIGURE A.2 – Interface d'authentification utilisateur

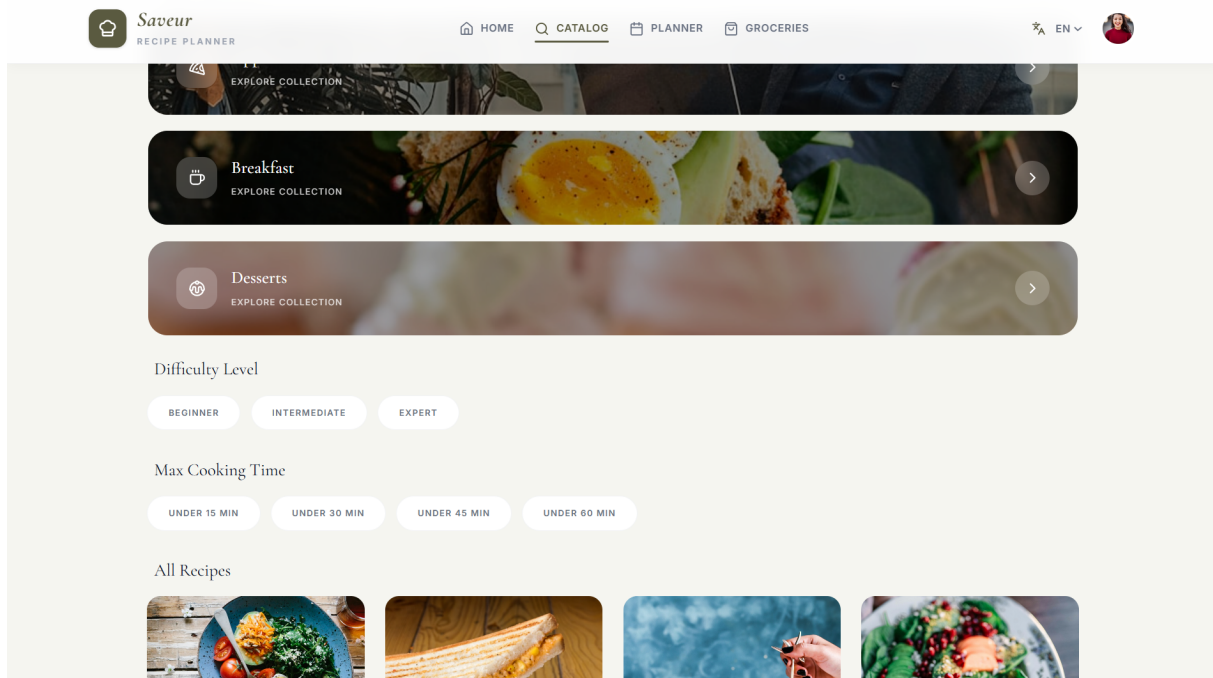


FIGURE A.3 – Catalogue de recettes avec collections et filtres

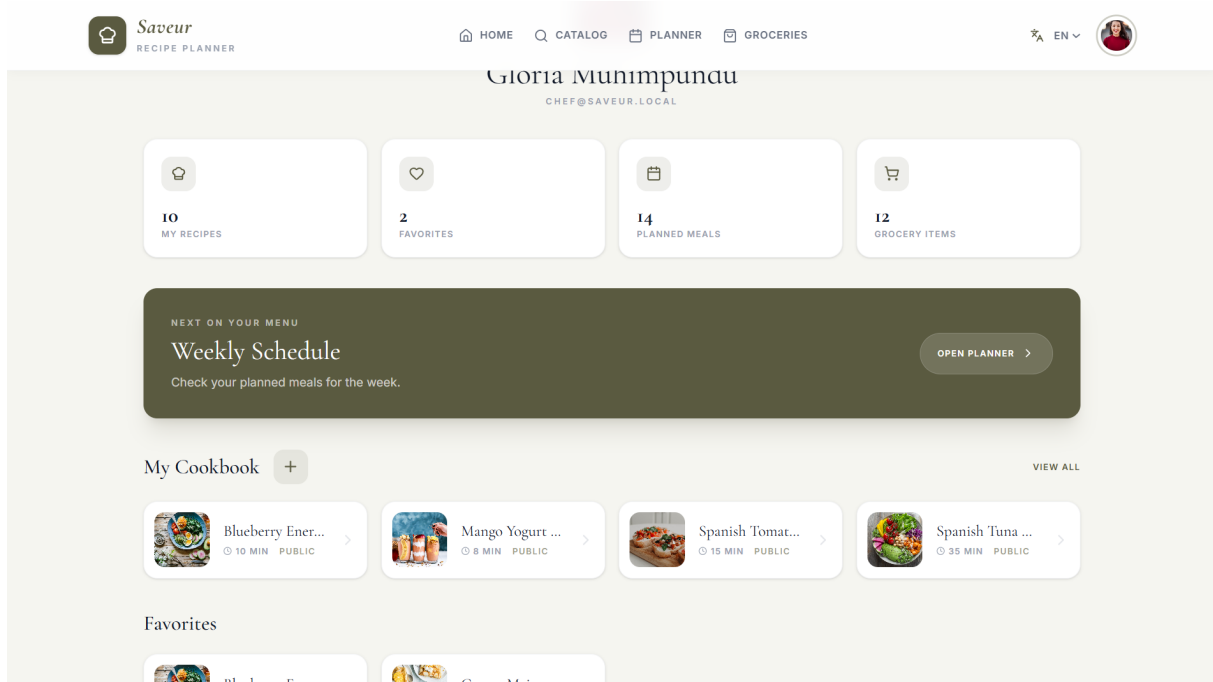


FIGURE A.4 – Profil et tableau de bord de l'utilisateur authentifié

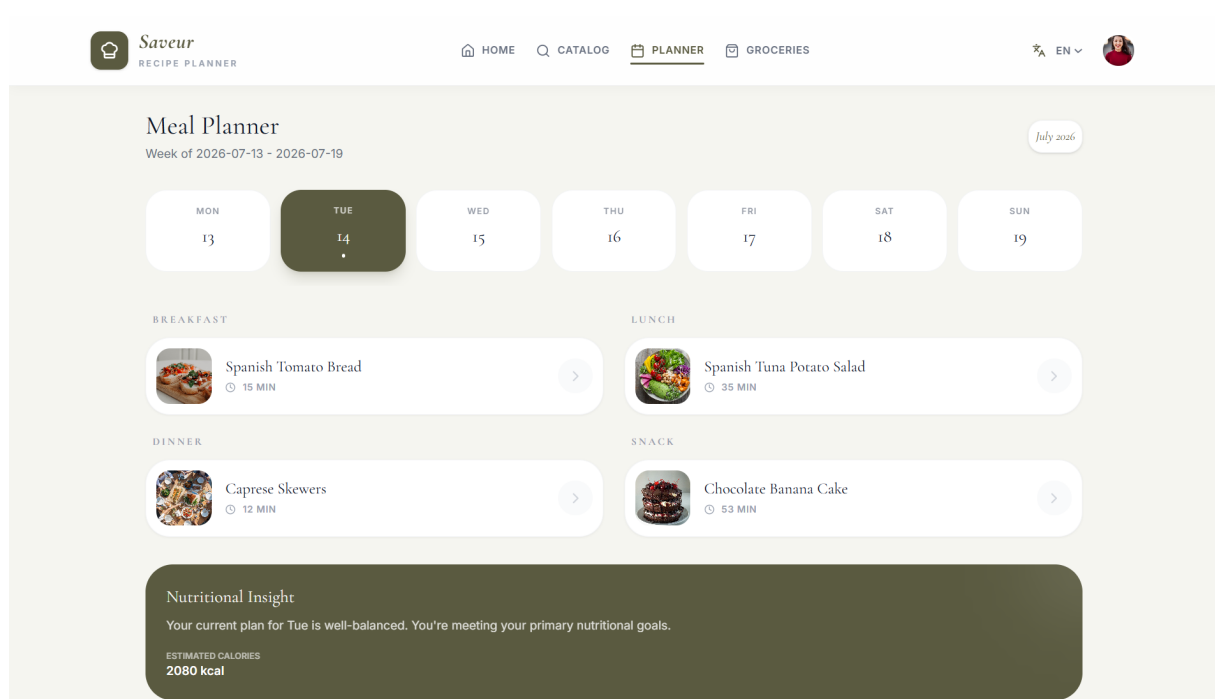


FIGURE A.5 – Planning hebdomadaire des repas

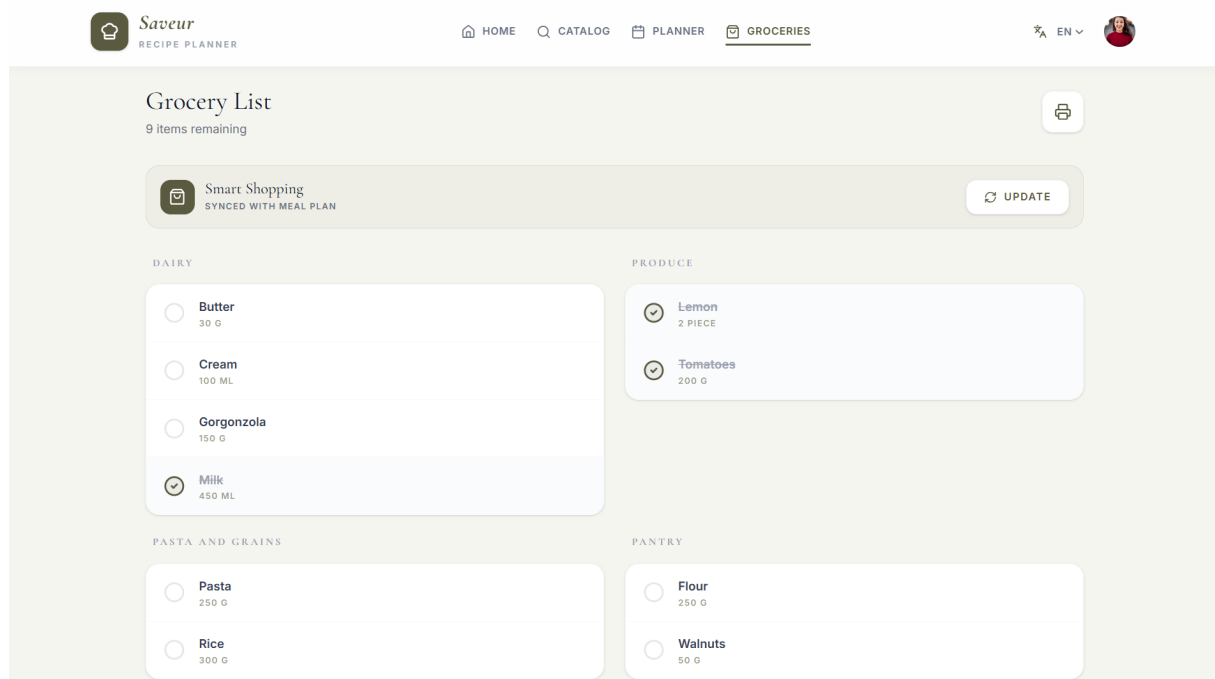


FIGURE A.6 – Liste de courses intelligente regroupée par rayons

Saveur
RECIPE PLANNER

HOME CATALOG PLANNER GROCERIES

EN

UPLOAD PHOTO TAKE PHOTO CLEAR IMAGE

Classic Chocolate Chip Cookies

/uploads/recipes/1783846136502-41ef6a5339b1175.webp

A timeless recipe for soft, chewy homemade chocolate chip cookies with perfectly golden edges and gooey, melted chocolate in every single bite.

Ingredients

Unsalted butter (softened) 1 cup

ADD INGREDIENT

Preparation Steps

STEP 1

Prep Oven

Preheat oven to 375°F (190°C) and line a baking sheet with parchment paper.

PREP 15 COOK 10

LEVEL 1 SERVINGS 24

CATEGORY Desserts

CUISINE International

☒ Visible in the public catalog

Save Recipe

FIGURE A.7 – Formulaire de création d’une recette avec image, ingrédients et étapes

Saveur
RECIPE PLANNER

HOME CATALOG PLANNER GROCERIES

EN

APPETIZERS By Demo Chef Spanish

Spanish Tomato Bread

Toasted bread rubbed with tomato, olive oil and salt.

15 min Level 1/5 4 servings

INGREDIENTS STEPS

What you'll need 4 SERVINGS

Bread 8 slice

Tomatoes 300 g

Olive oil 30 ml

Lemon 1 piece
OPTIONAL

Start Cooking Mode

FIGURE A.8 – Détail d’une recette avec ingrédients et accès au mode cuisine

Annexe B

Guide d'installation et d'exécution

Prérequis

- Node.js installé.
- MongoDB Server installé et démarré localement.
- Git pour récupérer le dépôt.
- Un terminal PowerShell, Git Bash ou équivalent.

Étapes

1. Installer les dépendances : `npm install`.
2. Copier `.env.example` vers `.env`.
3. Configurer `PORT`, `MONGO_URI`, `JWT_SECRET`, `CLIENT_URL`, `AUTO_SEED` et `VITE_API_URL`.
4. Vérifier que MongoDB fonctionne sur `127.0.0.1:27017`.
5. Lancer l'API avec `npm run api`.
6. Dans un autre terminal, lancer le frontend avec `npm run dev`.
7. Ouvrir `http://localhost:3000`. Si ce port est occupé, Vite peut proposer un autre port.
8. Vérifier le build avec `npm run build` et les tests avec `npm run test`.

Compte de démonstration

Le seed local crée un compte de démonstration destiné aux tests : `chef@saveur.local` avec le mot de passe `saveur123`. Ce compte ne doit pas être utilisé comme identifiant réel en production.

Annexe C

Principales routes API

Le tableau 3.2 dans le chapitre de conception contient l’inventaire principal des routes. Pour la soutenance, nous recommandons de démontrer au minimum : connexion, liste des recettes, création de recette, ajout au planning, génération de courses et récupération du dashboard.

Annexe D

Répartition des tâches

Les informations de groupe ne sont pas disponibles dans le dépôt. Le tableau suivant doit être complété avant la remise.

TABLE D.1 – Répartition des tâches du groupe

Étudiant	Contributions principales	Contributions secondaires	Tests/documentation
[À compléter : étudiant 1]	Backend, modèles, routes	Aide frontend	Tests API
[À compléter : étudiant 2]	Interface React, UX	Traductions	Captures
[À compléter : étudiant 3]	MongoDB, seed, liste de courses	Planning	Rapport
[À compléter : étudiant 4 si applicable]	Intégration	Corrections	Relecture

Annexe E

Table de traçabilité interne

TABLE E.1: Correspondance entre affirmations du rapport et preuves du dépôt

Affirmation	Preuve dépôt	Statut
Authentification JWT	server/utils/tokens.js, server/middleware/auth.js	Vérifié
Hashage bcrypt	server/controllers/authController.js	Vérifié
Sept collections Mongo-DB	server/models/*.js	Vérifié
Recettes avec ingrédients et étapes	server/models/Recipe.js	Vérifié
Catalogue filtrable	recipeController.js, Catalog.tsx	Vérifié
Planning hebdomadaire	MealPlan.js, Plan.tsx	Vérifié
Liste de courses intelligente	groceryListService.js, Groceries.tsx	Vérifié
PWA de base	public/manifest.webmanifest, public/sw.js	Vérifié
Tests automatisés	server/services/groceryListServiceUnitTest.js	Vérifié
Édition complète des recettes	Aucun endpoint d'édition trouvé	Manquant